

AI 활용 기술 문서

AI 도구 · 프롬프트 · 활용 내역 정리

Unity 멀티플레이 파티게임 「Stab in the Back」 (가제: 마검탈출맵 리메이크)

항목	내용
프로젝트명	(가제) 마검탈출맵 파티게임 — 코드네임 stab in the back
장르 / 플랫폼	1인칭 병맛 멀티플레이 탈출 파티게임 / PC
개발 환경	Unity 6 (URP 17.3), 새 Input System, Mirror 96.10.3, Blender 5.0
작성자	방유빈 (GitHub: DevLucidDreamer)
작성일	2026년 8월 9일
대상 기간	2026년 7월 12일 ~ 2026년 8월 9일
문서 목적	개발 과정에서 사용한 AI 도구와 프롬프트, 활용 내역 및 검증 절차 정리

1. 문서 개요

본 문서는 Unity 기반 멀티플레이 파티게임 「Stab in the Back」의 개발 과정에서 생성형 AI를 어떤 도구로, 어떤 방식의 프롬프트로, 어떤 작업에 활용했는지를 정리한 기술 문서다. 마인크래프트 «마검탈출맵» 장르를 1인칭 3D 게임으로 리메이크하는 프로젝트로, 일상 사물(국자·냉동참치 등)을 한 방 즉사 무기 "마검"으로 삼아 협동과 배신이 동시에 벌어지는 멀티플레이 탈출 게임이다.

개발은 1인 개발 체제로 진행되었으며, AI는 코드를 대신 작성해 주는 자동 완성 도구가 아니라 "설계 문서 → 구현 → 에디터 자동화 → 검증"의 각 단계를 함께 수행하는 페어 프로그래밍 파트너로 사용했다. 모든 산출물은 Unity 에디터 및 실제 플레이 테스트를 통해 사람이 최종 검증한 뒤 저장소에 반영했다.

2. 사용한 AI 도구

도구	형태 / 모델	주요 용도
Claude Code	CLI / 데스크톱 에이전트 (Anthropic Claude)	프로젝트 저장소에 직접 접근하는 코딩 에이전트. C# 게임 플레이 스크립트, Unity 에디터 확장, Python 유틸리티 작성 및 리팩터링, 기획·설계 문서 작성, 버그 원인 분석에 사용.
Blender MCP 연동	MCP 서버 (Claude Code 확장)	실행 중인 Blender 씬을 AI가 직접 조회·조작. 무기·캐릭터 로우폴리 모델링 작업의 상태 확인과 스크립트 기반 편집에 사용.
웹 검색 / 문서 조회	AI 내장 검색·페이지 읽기	Mirror 네트워킹 API, Unity 6 URP 변경점, Blender 5.0 파일 포맷 등 최신 레퍼런스 조사. 장르(마검탈출맵) 사전 조사 및 출처 정리.

※ 모델·에셋 제작 자체는 Blender에서 사람이 직접 수행했으며, AI는 파일 포맷 해석·임포트 파이프라인·프리팹 생성 자동화를 담당했다. 게임에 사용된 이미지·폰트·사운드는 AI로 생성하지 않았다.

3. 개발 영역별 AI 활용 내역

영역	활용 내용	산출물
기획 · 설계 문서	장르 조사 결과 정리, 게임 컨셉·재미 메커니즘 구조화, 개발 로드맵과 페이지별 완료 기준 정의	docs/기획서.md, 기획서2.md, 기획서_대기실.md, 개발_로드맵.md
싱글플레이 코어	1인칭 이동·시야·달리기·점프·물체 밀기, 레이캐스트 상호작용, 서랍/문 열기, 무기 스윙 연출, 준비물 체크리스트 UI 구현	PlayerController.cs, PlayerInteraction.cs, Openable.cs, Weapon.cs, ItemChecklist.cs 등
멀티플레이 전환	Mirror 도입을 Phase 0~7로 분할해 단계별 설계·구현. 권한 분리, 서버 검증, 상태 동기화 (SyncVar/Command/ClientRpc) 적용	NetworkPlayerSetup.cs, NetworkOpenable.cs, LobbyManager.cs, docs/multiplayer_phase0~7_setup.md
에디터 자동화	메뉴 한 번으로 맵·씬·프리팹을 절차적으로 생성하는 Unity 에디터 확장 다수	Editor/HouseBuilder.cs, Stage2Builder.cs, LobbyBuilder.cs, TitleSceneSetup.cs,

영역	활용 내용	산출물
	작성 (수작업 씬 구성 제거)	WeaponPrefabBuilder.cs 등 25종
에셋 파이프라인	Blender 미설치 환경에서 .blend를 직접 파싱해 Unity용 JSON 메시로 변환하는 도구 개발	Tools/blend_import/blendparse.py, blend2json.py, README.md
UI · 연출	타이틀 화면 구성(호스트/참가/종료 흐름), 카메라 드리프트·모닥불 깜빡임·반딧불 등 분위기 연출, UI 그라디언트	MainMenu.cs, TitleCameraDrift.cs, TitleFireFlicker.cs, TitleFireflies.cs, UIGradient.cs
폰트 · 로컬라이징	한글 TMP(TextMesh Pro) 폰트 애셋 생성 자동화 및 글리프 누락 대응	Editor/JalnanFontAssetBuilder.cs, Assets/Fonts/Jalnan2
디버깅	URP 렌더링 이상, 서랍 슬라이드 방향 오류, .blend 프리뷰 импорт 문제 등 원인 분석 및 수정	커밋 "렌더링 문제 해결" 외 다수

4. 프롬프트 활용 방식

4.1 프롬프트 작성 원칙

- 맥락 우선 — 엔진 버전(Unity 6 URP), 네트워킹 라이브러리(Mirror), 기존 스크립트 구조를 먼저 제시한 뒤 요구사항을 전달한다.
- 작업 단위 분할 — "멀티플레이 붙여줘" 같은 큰 요구 대신 Phase 0~7 로 쪼개고, 각 페이지마다 완료 기준(Definition of Done)을 명시한다.
- 문서 선행 — 구현 전에 설계 문서(.md)를 먼저 작성하게 하고, 문서를 검토·수정한 뒤 그 문서를 근거로 코드를 작성하게 한다.
- 검증 방법 동봉 — "무엇을 어떻게 눌러 확인하는지"까지 함께 요구해, 결과를 곧바로 플레이 테스트로 확인할 수 있게 한다.
- 제약 명시 — 기존 파일 구조 유지, 새 패키지 추가 금지, 주석 밀도·명명 규칙 등 프로젝트 규칙을 프롬프트에 포함한다.

4.2 실제 사용 프롬프트 예시

아래는 개발 과정에서 실제로 사용한 프롬프트를 작업 단위별로 요약·정리한 것이다.

단계	프롬프트 요지	결과물 / 조치
장르 조사	"마인크래프트 마검탈출맵 장르를 조사해서, 이 장르가 어떤 재미 메커니즘으로 굴러가는지와 맵 구성 방식을 출처와 함께 정리해줘."	재미 메커니즘 5가지(희소성=권력, 배신, 신경전, 탐색 보상, 경쟁 목표)로 구조화된 기획서 초안. 출처 링크 포함.
기획 구체화	"조사 결과를 바탕으로 '캠핑 가기 전 집에서 준비물 챙기기' 테마의 1인칭 파티게임 기획서를 작성해줘. 인원·조작·승리 조건까지 표로."	docs/기획서.md — 게임 개요표, 컨셉 한 줄 요약, 준비물 8종 목록, 조작 정의.
맵 자동 생성	"집 맵을 수작업으로 배치하지 않고 에디터 메뉴에서 절차적으로 생성하고 싶다. Tools > House > Build/Clear House 메뉴로 만들어줘."	Editor/HouseBuilder.cs — 내부 32×24m 집(거실·주방·침실·서재·욕실), 서랍 6개, 냉장고, 무기 배치를 메뉴 한 번으로 생성.
멀티 전환 설계	"지금 싱글 기준 코드를 Mirror로 옮기려 한다. 한 번에 고치지 말고 Phase 0부터 나눠서, 각 단계 완료 기준과 테스트 방법을 문서로 먼저 써줘."	multiplayer_phase0~7_setup.md 8개 문서 — 단계별 목표, 변경 파일, 검증 절차 정의.
권한 분리	"로컬 플레이어만 입력을 받도록 PlayerController와 상호작용에 훅을 추가해줘. 기존 싱글 동작은 깨지지 않게."	SetInputEnabled / Pitch / CameraTransform / SetRemotePitch 훅 추가 — 싱글·멀티 양쪽에서 동작.
타이틀 화면	"NetworkManager를 중복 생성하지 않으면서 타이틀에서 호스트/참가를 선택해 게임 씬으로 넘어가는 구조를 설계해줘."	GameLaunch(정적 상태) → NetworkAutoLaunch 부트스트랩 구조. Tools > Title > Setup Main Title 메뉴로 씬 자동 구성.
에셋 파이프라인	"이 PC에 Blender가 없어서 .blend가 프리뷰로만 임포트된다. Blender 없이 .blend를 직접 읽어 Unity 메시로 변환하는 스크립트를 만들어줘."	Tools/blend_import/ — zstd 해제, Blender 5.0 헤더·SDNA 해석, 좌표계 변환까지 수행하는 Python 파서.
버그 수정	"서랍이 반대 방향으로 열린다. Openable의 슬라이드 축 계산을 로컬 기준으로 다시 확인해줘."	슬라이드 방향 계산 수정. 동일 방식으로 URP 렌더링 이상, 폰트 글리프 누락 문제 처리.

※ 위 프롬프트는 실제 대화 내용을 작업 단위로 요약한 것으로, 표현은 원문과 다를 수 있다.

5. 대표 활용 사례

5.1 Blender 5.0 .blend 파일 직접 파싱 도구 개발

문제 — Unity는 .blend를 임포트할 때 로컬에 설치된 Blender를 호출해 FBX로 변환한다. 개발 PC에 Blender가 설치되어 있지 않아 무기 모델 10종이 모델이 아닌 썸네일(PreviewImporter)로만 들어왔다.

AI 활용 — Blender 5.0의 파일 컨테이너 구조(zstd 압축, BLENDER17-01v0502 헤더, 32바이트 BHead, SDNA 타입 데이터베이스)를 조사하게 하고, 이를 근거로 Python 파서를 작성하도록 했다. 신규 AttributeStorage에서 position.corner_vert.material_index.UVMap.sharp_face를 추출하고, Blender의 Z-up 우수 좌표계를 Unity의 Y-up 좌수 좌표계로 변환((x, y, z) → (x, z, y), 와인딩 반전)하는 부분까지 포함한다.

결과 — Blender 설치 없이 .blend → JSON → Unity Mesh/Material/프리팹 파이프라인이 완성되었다. 변환 결과는 저장소에 커밋되어 다른 PC에서도 추가 설치 없이 빌드가 가능하다. 한계(정적 메시 전용, 스키닝·애니메이션 미지원)도 README에 함께 명시했다.

5.2 Mirror 멀티플레이 단계적 전환

싱글플레이 기준으로 완성된 이동·상호작용·무기·체크리스트 시스템을 멀티플레이로 옮기는 작업을, 한 번에 개조하지 않고 Phase 0~7로 분할했다. 각 페이지마다 AI에게 (1) 목표, (2) 변경 대상 파일, (3) 완료 기준, (4) ParrelSync 기반 테스트 절차를 담은 설계 문서를 먼저 작성하게 하고, 문서를 검토한 뒤 구현에 착수했다.

- Phase 0~2 — NetworkManager 부트스트랩, 플레이어 스폰, 로컬 권한 입력 분리
- Phase 3~5 — 상호작용 오브젝트 서버 권한 처리, 체크리스트 공유, 플레이어 색상 배정
- Phase 6~7 — LAN 방 탐색(비콘/광고), 방 코드, 호스트 인원 설정, 사망 시 래그돌 시체 연출

이 방식의 이점은 각 단계가 독립적으로 검증 가능하다는 점이다. 문제가 생기면 직전 페이지 문서와 커밋으로 되돌아가 원인을 좁힐 수 있었다.

5.3 에디터 확장을 통한 씬 구성 자동화

씬을 손으로 배치하는 대신, AI에게 "메뉴 한 번으로 다시 만들 수 있는" 에디터 스크립트를 작성하게 했다. 집 맵, 캠핑장(Stage 2), 로비, 타이틀 화면, 무기 프리팹, 래그돌, 플레이어 애니메이터 세팅이

모두 Tools 메뉴에서 재생성된다.

그 결과 씬 파일이 손상되거나 구조를 바꿔야 할 때 재작업 비용이 거의 들지 않고, 맵 구성이 코드로 남아 변경 이력을 Git으로 추적할 수 있다는 부수 효과를 얻었다.

6. 검증 및 품질 관리 절차

단계	수행 내용	담당
요구 정의	구현할 기능, 제약, 완료 기준 정의	사람
설계 문서화	설계 문서(.md) 초안 작성	AI → 사람 검토·수정
구현	스크립트 작성 및 기존 코드와의 통합	AI → 사람 코드 리뷰
컴파일 검증	Unity 에디터 컴파일 오류·경고 확인	사람 (Unity 에디터)
동작 검증	Play 모드 및 ParrelSync 다중 클라이언트 플레이 테스트	사람
반영	동작 확인 후 Git 커밋	사람

AI가 생성한 코드는 예외 없이 사람이 읽고 이해한 뒤 반영했으며, 이해되지 않는 코드는 설명을 요구하거나 직접 다시 작성했다. 특히 네트워크 권한 처리처럼 오류가 눈에 잘 띄지 않는 영역은 다중 클라이언트 실제 접속 테스트로 검증했다.

7. AI 활용 성과 요약

지표	수치	비고
프로젝트 자체 C# 스크립트	63개 / 약 10,500줄	런타임 38개, 에디터 확장 25개 (외부 라이브러리 제외)
설계·기획 문서	13개	docs/ 이하 기획서·로드맵·페이지별 셋업 문서
개발 기간	약 4주	2026-07-12 ~ 2026-08-09
자동 생성 가능한 씬/맵	5종	집 맵, 캠핑장, 로비, 타이틀, 무기 프리팹
신규 개발 도구	1종	Blender 미설치 환경용 .blend 파서 (Python)

정성적으로는 (1) 1인 개발에서 병목이 되는 반복 작업(씬 배치, 보일러플레이트, 문서화) 제거, (2) 익숙하지 않은 영역(Mirror 네트워킹, Blender 파일 포맷)의 학습 곡선 단축, (3) 설계 근거가 문서로 남아 작업 인계와 재개가 쉬워진 점을 성과로 꼽을 수 있다.

8. 한계 및 주의사항

- 환각 및 API 오류 — 존재하지 않는 Unity/Mirror API 를 제시하는 경우가 있어, 컴파일 검증 전까지는 결과를 신뢰하지 않았다. 특히 Unity 6-URP 17 처럼 최신 버전은 공식 문서 확인을 병행했다.
- 맥락 유실 — 대화가 길어지면 프로젝트 규칙을 잊는 경향이 있어, 설계 문서를 저장소에 남기고 매 작업마다 참조시키는 방식으로 보완했다.
- 네트워크 코드의 취약성 — 권한·동기화 관련 오류는 단일 클라이언트 테스트로는 드러나지 않으므로 반드시 다중 접속 테스트로 검증했다.
- 저작권 · 라이선스 — 게임에 사용된 폰트(Jalnan2), 외부 라이브러리(Mirror, Steamworks.NET)는 각 배포처 라이선스를 확인해 사용했다. AI 로 생성한 이미지·사운드 에셋은 사용하지 않았다.
- 책임 소재 — AI 산출물의 최종 책임은 개발자에게 있다는 전제로, 모든 코드는 사람이 검토한 뒤 커밋했다.

이 문서는 「Stab in the Back」 프로젝트의 실제 저장소 내용(커밋 이력, 문서, 스크립트)을 근거로 작성되었다.